```
# CELL 1 — Imports

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

import numpy as np
import yfinance as yf
from sklearn.preprocessing import StandardScaler
```

```
# CELL 2 — Download OHLCV (10 assets)

tickers = ["AAPL","MSFT","GOOG","AMZN","META","TSLA","NVDA","JPM","V","MA"]

df = yf.download(tickers, start="2010-01-01", auto_adjust=False)

opens  = df["Open"].values
highs  = df["High"].values
lows   = df["Low"].values
closes = df["Close"].values
vols   = df["Volume"].values
adjcls = df["Adj Close"].values

# [T, 10, 6]
data = np.stack([opens, highs, lows, closes, vols, adjcls], axis=2)

print("Raw data shape:", data.shape)
```

```
[**********100%**********]  10 of 10 completedRaw data shape: (4024, 10, 6)
```

```
# CELL 3 — Normalize using train-only statistics

flat = data.reshape(len(data), -1).astype(np.float32)

n_total = len(flat)
n_train = int(n_total * 0.8)
n_val   = int(n_total * 0.1)
n_test  = n_total - n_train - n_val

train_flat = flat[:n_train]
val_flat   = flat[n_train:n_train+n_val]
test_flat  = flat[n_train+n_val:]

scaler = StandardScaler()
scaler.fit(train_flat)

train_scaled = scaler.transform(train_flat)
val_scaled   = scaler.transform(val_flat)
test_scaled  = scaler.transform(test_flat)

train_data = train_scaled.reshape(-1, 10, 6)
val_data   = val_scaled.reshape(-1, 10, 6)
test_data  = test_scaled.reshape(-1, 10, 6)

print("Normalized shapes:", train_data.shape, val_data.shape, test_data.shape)
```

```
Normalized shapes: (3219, 10, 6) (402, 10, 6) (403, 10, 6)
```

```
# CELL 4 — Replace NaNs after scaling (detailed printout)

train_data = np.nan_to_num(train_data, nan=0.0)
val_data   = np.nan_to_num(val_data, nan=0.0)
test_data  = np.nan_to_num(test_data, nan=0.0)

print("NaNs in train_data:", np.isnan(train_data).sum())
print("NaNs in val_data:",   np.isnan(val_data).sum())
print("NaNs in test_data:",  np.isnan(test_data).sum())
```

```
NaNs in train_data: 0
NaNs in val_data: 0
NaNs in test_data: 0
```

```
# CELL 5 — Dataset for DeepFolio
```

```
class OHLCVDataset(Dataset):
    def __init__(self, data, input_len=96, pred_len=24):
        """
        data: [T, 10, 6] normalized OHLCV
        input_len: number of past timesteps (96)
        pred_len: forecast horizon (24)
        """
        self.data = data
        self.input_len = input_len
        self.pred_len = pred_len

    def __len__(self):
        return len(self.data) - self.input_len - self.pred_len

    def __getitem__(self, idx):
        # x_raw: [96, 10, 6]
        x_raw = self.data[idx : idx + self.input_len]

        # reshape to [10, 576] = 10 assets × (96 timesteps × 6 features)
        x = x_raw.transpose(1, 0, 2).reshape(10, -1)

        # y_raw: [24, 10, 6]
        y_raw = self.data[
            idx + self.input_len :
            idx + self.input_len + self.pred_len
        ]

        # target = Close price only → feature index 3
        # shape: [24, 10] → [10, 24]
        y = y_raw[:, :, 3].T

        return (
            torch.tensor(x, dtype=torch.float32),
            torch.tensor(y, dtype=torch.float32)
        )
```

```
# CELL 6 — DeepFolio Model (Stable)

class DeepFolio(nn.Module):
    def __init__(self, num_assets=10, seq_len=576, hidden=128, pred_horizon=24):
        super().__init__()

        self.num_assets = num_assets
        self.seq_len = seq_len
        self.pred_horizon = pred_horizon

        # 1. Per-asset temporal encoder (Conv1D)
        self.asset_encoder = nn.Sequential(
            nn.Conv1d(1, hidden, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.Conv1d(hidden, hidden, kernel_size=5, padding=2),
            nn.ReLU()
        )

        # 2. Cross-asset attention
        self.cross_attn = nn.MultiheadAttention(
            embed_dim=hidden,
            num_heads=4,
            batch_first=True
        )

        # 3. Temporal attention (assets as tokens)
        self.time_attn = nn.MultiheadAttention(
            embed_dim=hidden,
            num_heads=4,
            batch_first=True
        )

        # 4. Prediction head
        self.head = nn.Linear(hidden, pred_horizon)

    def forward(self, x):
        # x: [B, 10, 576]
        B = x.size(0)
```

```
        # Encode each asset independently
        x = x.reshape(B * self.num_assets, 1, self.seq_len)
        h = self.asset_encoder(x)          # [B*10, hidden, 576]
        h = h.mean(dim=2)                  # [B*10, hidden]

        # reshape back to [B, 10, hidden]
        h = h.reshape(B, self.num_assets, -1)

        # Cross-asset attention
        h, _ = self.cross_attn(h, h, h)

        # Temporal attention
        h, _ = self.time_attn(h, h, h)

        # Predict 24-step horizon
        out = self.head(h)                 # [B, 10, 24]

        return out
```

```
# CELL 7 — Build Dataloaders

input_len = 96
pred_len = 24

train_dataset = OHLCVDataset(train_data, input_len, pred_len)
val_dataset   = OHLCVDataset(val_data,   input_len, pred_len)
test_dataset  = OHLCVDataset(test_data,  input_len, pred_len)

batch_size = 32

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader   = DataLoader(val_dataset,   batch_size=batch_size, shuffle=False)
test_loader  = DataLoader(test_dataset,  batch_size=batch_size, shuffle=False)

print("Loaders ready.")
print("Train batches:", len(train_loader))
print("Val batches:",   len(val_loader))
print("Test batches:",  len(test_loader))
```

```
Loaders ready.
Train batches: 97
Val batches: 9
Test batches: 9
```

```
# CELL 8 — Training Loop

def train_model(model, train_loader, val_loader, epochs=30, lr=1e-3, device="cuda"):
    model = model.to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    mse_loss = nn.MSELoss()
    mae_loss = nn.L1Loss()

    print("Starting training...")

    for epoch in range(1, epochs + 1):
        model.train()
        train_mse = 0.0

        for x, y in train_loader:
            x, y = x.to(device), y.to(device)

            optimizer.zero_grad()
            pred = model(x)

            loss = mse_loss(pred, y)

            # Safety check
            if torch.isnan(loss):
                print(f"NaN detected at epoch {epoch}. Stopping early.")
                return

            loss.backward()
            optimizer.step()

            train_mse += loss.item()
```

```
        train_mse /= len(train_loader)

        # Validation
        model.eval()
        val_mse = 0.0
        val_mae = 0.0

        with torch.no_grad():
            for x, y in val_loader:
                x, y = x.to(device), y.to(device)
                pred = model(x)

                val_mse += mse_loss(pred, y).item()
                val_mae += mae_loss(pred, y).item()

        val_mse /= len(val_loader)
        val_mae /= len(val_loader)

        print(f"Epoch {epoch}/{epochs} | "
              f"Train MSE={train_mse:.6f} | "
              f"Val MSE={val_mse:.6f} | "
              f"Val MAE={val_mae:.6f}")

    print("Training complete.")
```

```
# CELL 9 — Initialize DeepFolio + Begin Training

model = DeepFolio(
    num_assets=10,
    seq_len=576,
    hidden=128,
    pred_horizon=24
)

train_model(
    model,
    train_loader,
    val_loader,
    epochs=30,
    lr=1e-3,
    device="cuda"
)
```

```
Starting training...
Epoch 1/30 | Train MSE=0.156921 | Val MSE=3.556948 | Val MAE=0.969643
Epoch 2/30 | Train MSE=0.109969 | Val MSE=3.957923 | Val MAE=1.094487
Epoch 3/30 | Train MSE=0.095877 | Val MSE=3.637199 | Val MAE=1.006873
Epoch 4/30 | Train MSE=0.085681 | Val MSE=3.991399 | Val MAE=1.113349
Epoch 5/30 | Train MSE=0.077213 | Val MSE=3.922252 | Val MAE=1.079322
Epoch 6/30 | Train MSE=0.062350 | Val MSE=3.597167 | Val MAE=0.955690
Epoch 7/30 | Train MSE=0.051298 | Val MSE=3.422168 | Val MAE=0.911491
Epoch 8/30 | Train MSE=0.052482 | Val MSE=4.245486 | Val MAE=1.188054
Epoch 9/30 | Train MSE=0.045429 | Val MSE=2.908252 | Val MAE=0.971554
Epoch 10/30 | Train MSE=0.043001 | Val MSE=4.106720 | Val MAE=1.579425
Epoch 11/30 | Train MSE=0.037843 | Val MSE=4.131263 | Val MAE=1.622981
Epoch 12/30 | Train MSE=0.034720 | Val MSE=4.833738 | Val MAE=1.859111
Epoch 13/30 | Train MSE=0.031789 | Val MSE=4.268090 | Val MAE=1.683715
Epoch 14/30 | Train MSE=0.031613 | Val MSE=4.085438 | Val MAE=1.701815
Epoch 15/30 | Train MSE=0.029497 | Val MSE=2.525664 | Val MAE=1.139934
Epoch 16/30 | Train MSE=0.028091 | Val MSE=3.768679 | Val MAE=1.565866
Epoch 17/30 | Train MSE=0.029281 | Val MSE=3.387285 | Val MAE=1.464841
Epoch 18/30 | Train MSE=0.028980 | Val MSE=3.431555 | Val MAE=1.554925
Epoch 19/30 | Train MSE=0.027429 | Val MSE=3.710384 | Val MAE=1.642253
Epoch 20/30 | Train MSE=0.028051 | Val MSE=3.651156 | Val MAE=1.578870
Epoch 21/30 | Train MSE=0.030675 | Val MSE=5.390244 | Val MAE=2.053176
Epoch 22/30 | Train MSE=0.029324 | Val MSE=3.159116 | Val MAE=1.442625
Epoch 23/30 | Train MSE=0.028704 | Val MSE=3.270621 | Val MAE=1.438682
Epoch 24/30 | Train MSE=0.026357 | Val MSE=3.102141 | Val MAE=1.375519
Epoch 25/30 | Train MSE=0.026223 | Val MSE=3.350803 | Val MAE=1.434684
Epoch 26/30 | Train MSE=0.026778 | Val MSE=3.578615 | Val MAE=1.589642
Epoch 27/30 | Train MSE=0.027621 | Val MSE=3.674064 | Val MAE=1.626557
Epoch 28/30 | Train MSE=0.024543 | Val MSE=4.018304 | Val MAE=1.702217
Epoch 29/30 | Train MSE=0.023002 | Val MSE=3.392048 | Val MAE=1.503164
Epoch 30/30 | Train MSE=0.026027 | Val MSE=3.277477 | Val MAE=1.456119
Training complete.
```

```
# CELL 10 — Test-Set Evaluation

def evaluate_test_set(model, test_loader, device="cuda"):
```

```
    model = model.to(device)
    model.eval()

    mse_loss = nn.MSELoss()
    mae_loss = nn.L1Loss()

    test_mse = 0.0
    test_mae = 0.0

    with torch.no_grad():
        for x, y in test_loader:
            x, y = x.to(device), y.to(device)
            pred = model(x)

            test_mse += mse_loss(pred, y).item()
            test_mae += mae_loss(pred, y).item()

    test_mse /= len(test_loader)
    test_mae /= len(test_loader)

    print("Final Test Evaluation:")
    print(f"Test MSE = {test_mse:.6f}")
    print(f"Test MAE = {test_mae:.6f}")

# Run evaluation
evaluate_test_set(model, test_loader, device="cuda")
```

```
Final Test Evaluation:
Test MSE = 45.803207
Test MAE = 5.075695
```

```
# CELL 4R — Train/Val/Test Split for Return Pipeline

# 'data' must already be loaded as your full OHLCV numpy array
# shape: [T, num_assets, features]

T = len(data)

train_end = int(T * 0.8)
val_end   = int(T * 0.9)

train_data = data[:train_end]
val_data   = data[train_end:val_end]
test_data  = data[val_end:]

print("Data split complete.")
print("Train:", train_data.shape)
print("Val:",   val_data.shape)
print("Test:",  test_data.shape)
```

```
Data split complete.
Train: (3219, 10, 6)
Val: (402, 10, 6)
Test: (403, 10, 6)
```

```
# CELL 5R — Return-Based Dataset (log returns)

from torch.utils.data import Dataset, DataLoader

class ReturnDataset(Dataset):
    def __init__(self, data, input_len, pred_len):
        """
        data: numpy array of shape [T, num_assets, features]
        We assume Close is at index 3 in the OHLCV ordering.
        """
        self.data = data
        self.input_len = input_len
        self.pred_len = pred_len

        # Extract Close prices
        close = data[:, :, 3]  # shape [T, num_assets]

        # Compute log returns
        log_ret = np.log(close[1:] / close[:-1])  # shape [T-1, num_assets]

        # Align features (drop first row to match log_ret length)
        self.features = data[1:]  # shape [T-1, num_assets, features]
```

```
        self.targets = log_ret    # shape [T-1, num_assets]

    def __len__(self):
        return len(self.targets) - self.input_len - self.pred_len

    def __getitem__(self, idx):
        x = self.features[idx : idx + self.input_len]              # [input_len, num_assets, features]
        y = self.targets[idx + self.input_len : idx + self.input_len + self.pred_len]  # [pred_len, num_assets]

        # Transpose to match DeepFolio: [assets, seq]
        x = torch.tensor(x, dtype=torch.float32).permute(1, 0, 2)  # [num_assets, input_len, features]
        y = torch.tensor(y, dtype=torch.float32).permute(1, 0)     # [num_assets, pred_len]

        return x, y
```

```
# CELL 7R — Return-Based Dataloaders

input_len = 96
pred_len = 24
batch_size = 32

# Build datasets using log returns
train_dataset = ReturnDataset(train_data, input_len, pred_len)
val_dataset   = ReturnDataset(val_data,   input_len, pred_len)
test_dataset  = ReturnDataset(test_data,  input_len, pred_len)

# Build dataloaders
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader   = DataLoader(val_dataset,   batch_size=batch_size, shuffle=False)
test_loader  = DataLoader(test_dataset,  batch_size=batch_size, shuffle=False)

print("Return loaders ready.")
print("Train batches:", len(train_loader))
print("Val batches:",   len(val_loader))
print("Test batches:",  len(test_loader))
```

```
Return loaders ready.
Train batches: 97
Val batches: 9
Test batches: 9
```

```
# CELL 9R
model = DeepFolio(
    num_assets=10,
    seq_len=576,   # 96 time steps * 6 features
    hidden=128,
    pred_horizon=24
)

train_model(
    model,
    train_loader,
    val_loader,
    epochs=30,
    lr=1e-3,
    device="cuda"
)
```

```
Starting training...
NaN detected at epoch 1. Stopping early.
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.